Karunya INSTITUTE OF TECHNOLOGY AND SCIENCES
(Declared as Deemed to be University under Sec.3 of the UGC Act, 1956)
A CHRISTIAN MINORITY RESIDENTIAL INSTITUTION
AICTE Approved & NAAC Accredited

A SKILL BASED EVALUATION REPORT

SUBMITTED BY

[STUDENT NAME] ([REG. NO])

COURSE CODE
24AI2003

COURSE NAME

INTELLIGENT ROBOTICS

OCTOBER 2026

DIVISION OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

INTELLIGENT 5-AXIS CNC MACHINE WITH TOOL-CENTRE-POINT KINEMATICS, PID SERVO CONTROL, MACHINE-LEARNING TOOL MONITORING AND VISION-GUIDED PROBING

Submitted by

[STUDENT NAME] ([REG. NO])



DIVISION OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

KARUNYA INSTITUTE OF TECHNOLOGY AND SCIENCES
(Declared as Deemed-to-be-under Sec-3 of the UGC Act, 1956) Karunya Nagar, Coimbatore - 641 114. INDIA

OCTOBER 2026

ABSTRACT

A five-axis CNC machine can keep its cutting tool perpendicular to a curved surface, which gives a better surface finish and reaches features that a three-axis machine cannot. Doing this well needs the main skills of intelligent robotics: kinematics, motion planning, feedback control, sensing, perception and machine learning. This project designs and simulates a table-tilting 5-axis milling machine (linear axes X, Y, Z; tilt axis A; rotary table C) and adds four intelligent subsystems, each compared with a simpler baseline.

(1) Kinematics and planning: closed-form forward and inverse kinematics with tool-centre-point compensation (round-trip error 1.4×10^{-14} mm), a spiral finishing path over a dome with the tool normal to the surface, a tool-holder collision check, and a feed-rate planner that respects every joint's speed and acceleration limits. At a constant feed, one axis would need 11.0 times its top speed near the kinematic singularity; the planner removes this for a 3.4% longer cycle. The program is written as G93 inverse-time G-code and verified by reading it back. (2) Servo control: each axis is modelled with mass, friction, gravity and cutting load, and controlled by PID with feedforward tuned by pole placement. The RMS contour error falls from 399.2 µm (P) to 20.7 µm (PID + feedforward at 25 Hz), and to 3.8 µm at 50 Hz bandwidth. (3) Tool monitoring: spindle current and vibration windows are turned into eight physical features; a random forest recognises normal, worn, chattering and broken-tooth states with 98.8% accuracy on unseen data with unknown work material, against 84.0% for hand-written rules, and drives adaptive speed and feed changes. (4) Part localisation: a camera with Otsu thresholding and image moments finds the part to 0.397 mm, and a touch probe with least-squares fitting refines this to 0.003 mm; without the camera the probe found the part in only 17% of placements.

An interactive 3D simulator shows the machine running the program, and servo firmware and a hardware guide prepare a later physical build. All results are produced by one script, so every number in this report can be regenerated.

Keywords: 5-axis CNC, inverse kinematics, singularity, feed-rate planning, G-code, PID, feedforward, pole placement, tool condition monitoring, random forest, decision tree, computer vision, touch probing, least squares.

PROBLEM STATEMENT

Curved parts such as moulds, turbine blades, impellers and medical implants are finished best with the tool held perpendicular to the surface. A three-axis machine cannot do this: its tool always points straight down, so on a curved surface a ball-end tool cuts with a different part of the ball at every point, and steep walls are reached poorly or not at all. A five-axis machine solves this by adding two rotary axes, but the two extra axes bring problems of their own:

- **Kinematics.** The CAM program describes the tool on the part, while the machine moves joints. Each tool direction has two joint solutions, and at one orientation (the singularity) the rotary table would have to spin very fast for a tiny move on the part.
- **Motion feasibility and safety.** A constant cutting feed can demand joint speeds and accelerations the motors cannot produce, and a tilted tool holder can collide with the part or the table.
- **Accuracy.** Gravity, friction and cutting forces push the axes off the path, and every micrometre of contour error shows on the finished surface.
- **Process state.** Tool wear, chatter and broken teeth ruin parts and tools, but are hard to see while the machine is cutting.
- **Part location.** A part clamped by hand is never exactly where the program expects it; a few millimetres of error makes the part scrap.

Problem. Design, implement and evaluate an intelligent 5-axis CNC system that (a) converts a tool path into safe, feasible joint motion; (b) follows it accurately with well-tuned servo control; (c) monitors the cutting process with machine learning and reacts to problems; and (d) locates the workpiece automatically with vision and probing. Every method should be explainable step by step, and the design should be ready for a later hardware build.

Objectives

1. Derive and verify forward and inverse kinematics of a table-tilting A-C machine, including solution choice and singularity handling (kinematics).
2. Generate a 5-axis finishing path, check holder collisions and joint limits, plan feed rates within joint speed and acceleration limits, and output verified G-code (motion planning).
3. Model the five servo axes and compare P, PID and PID + feedforward control tuned by pole placement; find the best bandwidth (control, optimisation).
4. Detect tool wear, chatter and broken teeth from spindle current and vibration with machine learning, compare it with fixed rules, and adapt the process (AI/ML, sensing).
5. Locate the workpiece with a camera and a touch probe and measure the accuracy of each step (perception).

6. Provide an interactive 3D simulator, servo firmware and a hardware guide (simulation, hardware readiness).

Why this is an intelligent robotics problem

A 5-axis machine tool is a serial robot with five joints, and the same questions arise as for a robot arm: how joint motion maps to the motion of the end effector (kinematics), how to move between poses smoothly and within the limits of the actuators (motion planning), how to make each joint follow its command despite disturbances (control), how to know where the work object is (perception), and how to recognise and respond to abnormal situations (machine learning and decision making). Working on a machine tool makes these questions concrete, because the result is measured in micrometres on the finished part.

Expected outcomes

- Verified kinematics and a feasible, collision-free 5-axis program in standard G-code.
- A servo control design with a clear tuning rule and a measured accuracy improvement over P control.
- A tool monitor that is more reliable than fixed thresholds and acts on what it detects.
- A localisation method that finds a misplaced part reliably and accurately.
- Simulation tools, firmware and documentation that allow the work to be demonstrated and built.

Scope and assumptions

The machine is desktop size (300 × 300 × 240 mm travel). The A and C axes are assumed to intersect at the centre of the table top. The case study is the finishing of a dome with a 3 mm ball-end tool, plus the localisation of a rectangular block. Machine dynamics, sensor signals and camera images are simulated from simple physical models; the limitations of this are discussed in the Conclusion.

METHODOLOGY / ARCHITECTURE

1. System architecture

The system has three chains (Figure 1). The planning chain turns the part geometry into G-code: CAM path, inverse kinematics, collision and limit checks, feed planning. The control chain executes it: the CNC controller interpolates the moves at 1 kHz and five servo drives make the axes follow, using encoder feedback. Two sensing chains add intelligence: vision and probing locate the part before cutting (giving the work offset), and the machine-learning monitor watches the cut and overrides the feed and spindle speed.

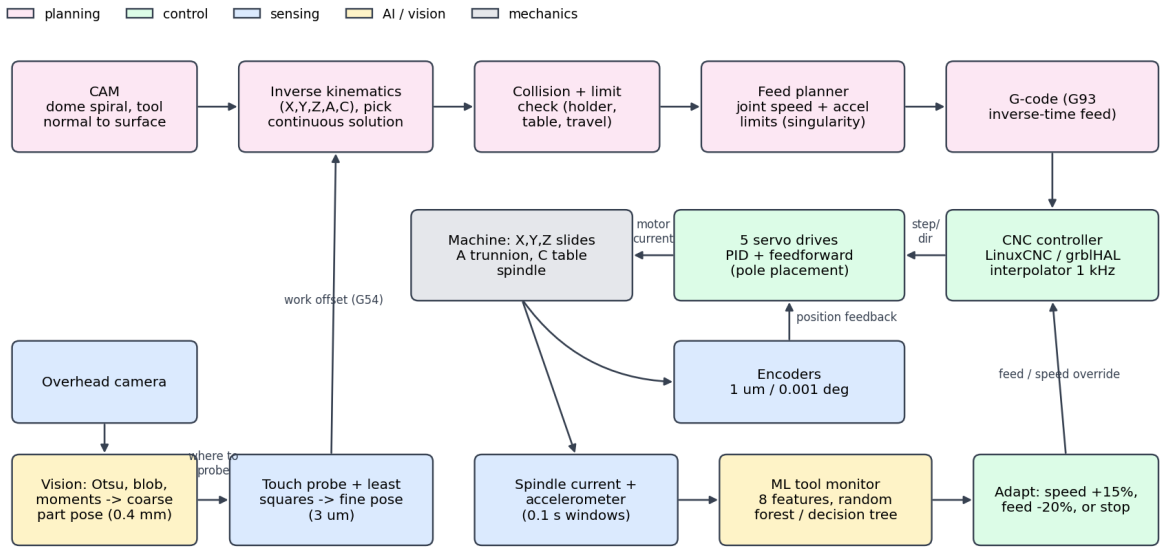


Figure 1: System architecture of the intelligent 5-axis CNC

2. Machine design

A table-tilting (trunnion) layout was chosen (Figure 2). The spindle moves in X, Y and Z and always points down along the machine Z axis. The table tilts about the X axis (A axis) and spins about its own axis (C axis), with C mounted on the A cradle. This layout is the most common for small 5-axis machines because the spindle stays rigid and vertical, and both rotary axes can use compact worm or harmonic drives. Its cost is that the part moves, so the linear axes must compensate whenever the table turns.

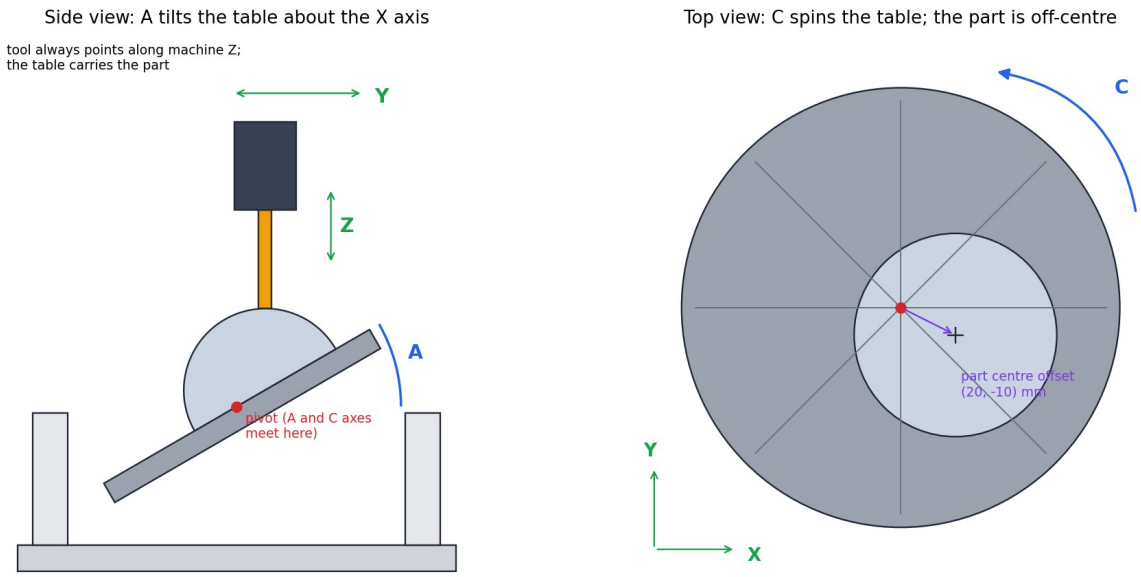


Figure 2: Machine axes: A tilts the table about X (left); C spins the table, and the part is clamped off-centre (right)

Table 1: Machine specification used in the simulation

Axis / part	Range and limits	Role
X, Y, Z slides	±150, ±150, ±120 mm; 50 mm/s; 500 mm/s²	Move the spindle; the tool always points along machine Z
A trunnion	−30° … +110°; 90 °/s; 720 °/s²	Tilts the table about X
C rotary table	continuous; 180 °/s; 1440 °/s²	Spins the part about the table axis
Spindle + ball-end tool	R 3 mm ball, 25 mm stick-out, Ø 24 mm holder	Cuts; holder geometry is used for collision checks
Encoders	1 µm (linear), 0.001° (rotary)	Position feedback for the servo loops
Process sensors	spindle current, accelerometer (10 kHz)	Inputs to the machine-learning tool monitor
Localisation sensors	overhead camera (0.4 mm/pixel), touch probe (2 µm)	Find the part before cutting

3. Software modules

#	Module (file)	Function
M1	Kinematics (kinematics.py)	Forward/inverse kinematics, two-solution choice, singularity handling, joint limits
M2	CAM (toolpath.py)	Dome spiral with tool normal to surface, holder collision check, G93 G-code writer and reader
M3	Motion planner (motion_planner.py)	Feed planning under joint speed and acceleration limits; simulator data
M4	Axis control (axis_control.py)	Servo model, interpolator, P / PID / PID + feedforward, pole placement, bandwidth sweep
M5	Tool monitor (tool_monitor.py)	Current and vibration simulation, 8 features, rules vs decision tree vs random forest, adaptive job
M6	Part finding (probing.py)	Otsu threshold, morphology, image moments; touch probe with least-squares edge fitting
M7	Simulator and firmware	3D browser simulator (sim/cnc_sim.html); one-axis servo firmware (STEP/DIR in, encoder, PID)
M8	Experiments (run_all.py)	Runs every module and writes all figures, metrics (JSON) and the G-code

4. Kinematics

4.1 Forward kinematics

Two coordinate frames are used: the machine frame (fixed to the machine base) and the part frame (fixed to the part, moving with the table). A point p fixed on the part is carried first by the C table and then by the A cradle, so in machine coordinates it is at $R\ p$ with

$$R(A,C) = R_x(A)\ R_z(C) \tag{1}$$

where $R_x(A)$ is a rotation by A about the x axis and $R_z(C)$ a rotation by C about the z axis. The tool tip is at the machine position $[X,Y,Z]^T$ and the tool points along machine z, $e_z = [0,0,1]^T$. Undoing the table rotation gives the tool tip and the tool direction on the part:

$$p = R^T [X,Y,Z]^T, \qquad u = R^T e_z = (\sin A\ \sin C,\ \ \sin A\ \cos C,\ \ \cos A)^T \tag{2}$$

4.2 Inverse kinematics

The CAM program gives p and u; the controller needs the joints. Equation (2) can be solved directly for the rotary axes:

$$A = \pm \arccos u_z, \qquad C = atan2(u_x,\ u_y) \tag{3}$$

(with C increased by 180° for the negative-A solution), and the linear axes follow from the forward relation:

$$[X,Y,Z]^T = R_x(A)\ R_z(C)\ p \tag{4}$$

Equation (4) is what makes the machine keep the tool on the same spot of the part while the table turns: if the part is off-centre, X and Y must move. This is called tool-centre-point (TCP) control. Three practical points are handled in the code:

- **Two solutions.** (A, C) and $(-A, C+180°)$ point the tool the same way. The solution inside the A limits and closest to the previous pose is kept, so the table never makes an unnecessary half-turn.
- **Unwrapping C.** C is continuous, so each new C is shifted by a multiple of 360° to be nearest to the previous value.
- **Singularity.** At $A=0$ the tool is vertical and u does not depend on C (the factor $\sin A$ in equation (2) is zero), so C is undefined and the previous C is kept. Near $A=0$, a small move on the part needs a large change of C, which the feed planner must slow down.

Table 2: Worked inverse-kinematics examples (from kinematics.py)

Tool tip p (mm)	Tool direction u	Joints (X, Y, Z in mm; A, C in degrees)
(20, -10, 28)	(0.00, 0.00, 1.00)	(20.00, -10.00, 28.00, 0.00, 0.00)
(10, 0, 20)	(0.00, 0.50, 0.87)	(10.00, -10.00, 17.32, 30.00, 0.00)
(20, 25, 10)	(0.71, 0.00, 0.71)	(-25.00, 7.07, 21.21, 45.00, 90.00)
(-15, 5, 12)	(-0.61, -0.61, 0.50)	(14.14, -6.86, 12.12, 60.00, -135.00)

The first row is the singular case (tool vertical): A = 0 and C stays at its previous value. Verification: for 1000 random poses, running inverse and then forward kinematics returns the same tool pose to within 1.4e-14 mm, which is the limit of floating-point arithmetic.

5. Toolpath and collision checking

The part is a dome: a cap of a sphere of radius $R_s=40$ mm whose centre is 15 mm below the table, standing 25 mm high and clamped with its centre 22 mm from the table centre (offset $o=(20,\ -10,\ 0)$ mm). A point on the dome at angle φ from its top and angle θ around it has the outward normal

$$n=(\sin\varphi\ \cos\theta,\quad \sin\varphi\ \sin\theta,\quad \cos\varphi)^T \tag{5}$$

The spiral path increases φ from 0 to 60° while θ makes 12 full turns, $\theta=24\pi\ \varphi/\varphi_{max}$, and φ is stepped so that consecutive points are about 0.5 mm apart on the surface. With a ball-end tool of radius $r=3$ mm, the programmed point is the centre of the ball and the tool axis is the surface normal:

$$S=c+R_s\ n,\qquad p=S+r\ n,\qquad u=n \tag{6}$$

where c is the sphere centre and S the contact point. The result is 2882 points. For collision checking, the tool holder (radius 12 mm) starts 25 mm up the tool. Sixteen points (x_k, y_k, z_k) on

the rim of its lower face are computed, and every one must lie above the part surface (or the table) underneath it:

$$\min_k \left(z_k - h\left(x_k, y_k\right) \right) > 0 \tag{7}$$

where $h(x, y)$ is the height of the dome, or zero on the table. Joint limits are checked at every point as well.

6. Feed-rate planning

The CAM program asks for a constant feed $F = 20$ mm/s along the part. For the move from point k to point k+1, of length d_k on the part and joint change $\Delta q_{i,k}$ for joint i, the planner first sets the time from the feed and then raises it so that no joint exceeds its top speed $v_{max,i}$:

$$t_k = \max_i \left(\frac{d_k}{F}, \quad \frac{|\Delta q_{i,k}|}{v_{max,i}} \right) \tag{8}$$

The joint velocities are then $v_{i,k} = \Delta q_{i,k} / t_k$. At each junction between two moves, the acceleration needed is

$$a_{i,k} = \frac{|v_{i,k+1} - v_{i,k}|}{(t_k + t_{k+1})/2} \tag{9}$$

If the worst ratio $\rho_k = \max_i a_{i,k} / a_{max,i}$ is above 1, both moves at that junction are stretched. Because acceleration scales with $1/t^2$, stretching by $\sqrt{\rho_k}$ brings the junction exactly to the limit:

$$t_k \leftarrow t_k \sqrt{\rho_k} \tag{10}$$

This is repeated until no junction exceeds its limit, with the machine starting and ending at rest. Finally the times are written as G-code in inverse-time feed mode (G93), the standard way to give a feed rate to moves that combine millimetres and degrees. F is the reciprocal of the move time in minutes, so with t_k in seconds:

$$F_{G93} = \frac{60}{t_k} \tag{11}$$

The G-code file is then read back and passed through forward kinematics to check that it reproduces the intended path.

7. Servo control and tuning

Each axis is a motor driving a mass m (or an inertia, for A and C) against viscous damping b, Coulomb friction F_c and a load F_L: gravity on Z and on the unbalanced A cradle, and a varying 15 N cutting force on X and Y. The control loop is shown in Figure 3.

$$m\ \ddot{x} = u - b\ \dot{x} - F_c\ \ sgn(\dot{x}) - F_L \tag{12}$$

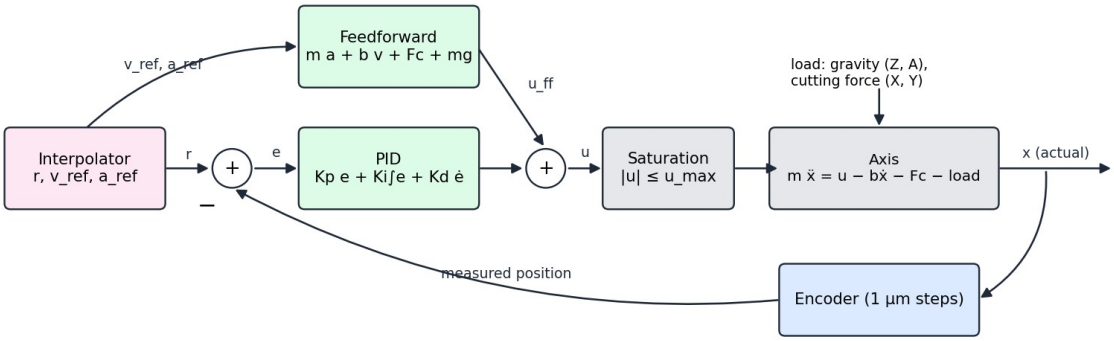


Figure 3: Servo control loop of one axis: PID on the error plus model-based feedforward

The interpolator joins the planned points with straight lines every 1 ms and smooths them with a 20 ms moving average, giving the reference position r, velocity v_{ref} and acceleration a_{ref}. With the error $e = r - x_{meas}$ (the encoder position, quantised to 1 µm or 0.001°), the controller output is

$$u = K_p e + K_i \int e\ \ dt + K_d \frac{de}{dt} + u_{ff} \tag{13}$$

$$u_{ff} = m\ \ a_{ref} + b\ \ v_{ref} + F_c\ \ sgn(v_{ref}) + m\ \ g \tag{14}$$

The feedforward term u_{ff} is the force that a perfect model says is needed to follow the reference (the gravity term *mg* is used on Z only). The PID part then only has to correct what the model does not know, mainly the cutting force. The integral is paused while the output is saturated (anti-windup), and the derivative is low-pass filtered so that it does not amplify encoder steps.

Tuning by pole placement. Ignoring the integral, the closed loop of equations (12) and (13) has the characteristic equation on the left of (15); matching it to a standard second-order system with natural frequency ω_n and damping ratio ζ gives the gains:

$$s^2 + \frac{b + K_d}{m} s + \frac{K_p}{m} = s^2 + 2\zeta\,\omega_n s + \omega_n^2 \tag{15}$$

$$K_p = m\,\omega_n^2, \qquad K_d = 2\zeta\,\omega_n m - b, \qquad K_i = \frac{K_p\,\omega_n}{10} \tag{16}$$

With $\zeta = 0.8$, a single number, the bandwidth ω_n, tunes all five axes. The integral gain is placed a decade below the bandwidth so that it removes steady errors without destabilising the loop. The next table lists the gains at $\omega_n = 2\pi \cdot 25$ rad/s; Section C of the Implementation compares bandwidths from 5 to 80 Hz.

Table 3: Axis models and pole-placement gains at 25 Hz (SI units: N, m or N m, rad)

Axis	m (kg or kg m²)	K_p	K_d	K_i
X	6	148,044	1,468	2,325,471
Y	6	148,044	1,468	2,325,471
Z	4	98,696	965	1,550,314
A	0.02	493	4.98	7,752
C	0.01	247	2.49	3,876

The quality measure is the contour error: the distance from the actual tool position on the part to the nearest point of the intended path. Unlike the following error of a single axis, it is what appears on the machined surface:

$$\varepsilon(t) = \min_{s} |\, p_{act}(t) - p_{ref}(s) |\qquad (17)$$

8. Machine-learning tool monitoring

The monitor turns two cheap signals into a decision every 0.1 s (Figure 4). Because no physical machine is available yet, the signals are generated from a simple physical model:

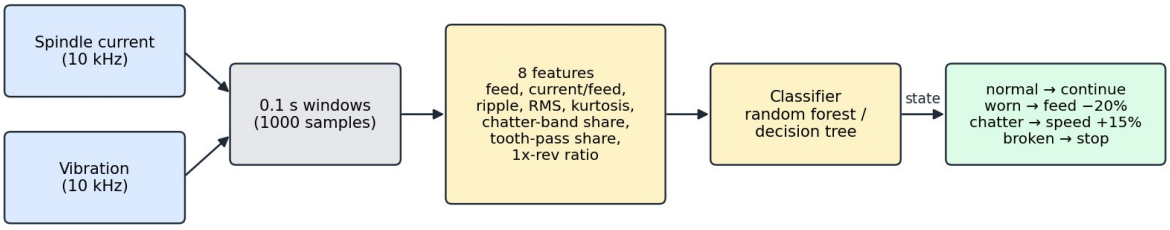


Figure 4: Tool-monitoring pipeline: signals → windows → features → classifier → action

- **Cutting force.** One pulse per tooth pass. For spindle speed n (rpm) and $z = 2$ flutes, the tooth-passing frequency is $f_{tp} = zn/60$. The force grows with the feed factor f, the wear w and an unknown material factor k_m between 0.7 and 1.5: $F(t) = 100\, k_m f (1 + 0.8\, w)\, max(0,\ \sin 2\pi f_{tp} t)$. A broken tooth removes every other pulse, and the remaining tooth cuts almost double.
- **Vibration.** The force passed through the machine's structural resonance (1.3–1.7 kHz), plus noise that grows with wear (rubbing of a blunt edge). Chatter adds a large self-excited oscillation at the resonance.
- **Spindle current.** Idle current (2 A) plus a term proportional to the average cutting force, plus sensor noise.

Each window of $N = 1000$ samples gives eight features that a machinist can interpret (next table). Two of them are defined by the usual formulas for a signal $x_1 \ldots x_N$ with mean $\bar{x}$ and standard deviation σ:

$$RMS = \sqrt{\frac{1}{N}\sum_{k=1}^{N} x_k^2}, \qquad kurtosis = \frac{1}{N\sigma^4}\sum_{k=1}^{N}\left(x_k - \bar{x}\right)^4 - 3 \tag{18}$$

Table 4: The eight features and what they respond to

#	Feature	Physical meaning
1	Feed (known)	Commanded chip load; lets the model separate feed from wear
2	Cutting current / feed	$\left(\bar{I} - I_{idle}\right)/f$: cutting force per unit feed; rises with wear (and with harder material)
3	Current ripple	Standard deviation of the current
4	Vibration RMS	Overall vibration level; rubbing of a worn edge raises it
5	Vibration kurtosis	Spikiness of the signal; the impacts of a broken tooth raise it
6	Chatter-band share	Share of vibration energy in 1.1–1.9 kHz, the resonance band
7	Tooth-pass share	Share of energy at f_{tp}, the normal cutting rhythm
8	1×rev / tooth-pass	Energy at the spindle frequency over energy at f_{tp}; large when one tooth is missing

Three classifiers were trained on 1200 windows (300 per class) and tested on 400 new windows from a different random seed: (i) hand-written thresholds on features 2, 6 and 8, the "no machine learning" baseline; (ii) a decision tree of depth 4; (iii) a random forest of 100 trees. A decision tree chooses, at every node, the feature and threshold that make the two branches as pure as possible, measured by the Gini impurity of the class proportions p_c in a node:

$$Gini = 1 - \sum_{c} p_c^2 \tag{19}$$

A random forest trains B such trees, each on a random resample of the data and with a random subset of features at each node, and takes their majority vote:

$$\hat{y}(x) = mode\left\{h_1(x),\ h_2(x),\ \ldots,\ h_B(x)\right\} \tag{20}$$

Averaging many trees lowers the variance of a single tree. In the adaptive job the monitor runs once per second and the controller reacts: chatter → spindle speed +15% (moving out of the unstable speed range); worn → feed −20% and a tool-change flag; broken tooth → feed hold and spindle stop.

9. Part localisation: vision and probing

9.1 Vision (coarse)

The overhead camera image (0.4 mm pixels, uneven light, bright metal chips, 0.5% scale calibration error) is thresholded with Otsu's method, which tries every threshold t and keeps the one that best separates the dark and the bright pixels (class weights ω_0, ω_1 and means μ_0, μ_1):

$$t^{*} = arg \max_{t} \omega_0(t)\,\omega_1(t)\left[\mu_0(t) - \mu_1(t)\right]^2 \tag{21}$$

Morphological opening (erosion then dilation) removes the chips, and the largest connected blob is taken as the part. Its centroid $\bar{q}$ gives the position, and the direction of its longest axis, the main eigenvector v of the second-moment (covariance) matrix of the blob's pixel positions q_k, gives the rotation:

$$\Sigma = \frac{1}{N}\sum_{k}\left(q_k - \bar{q}\right)\left(q_k - \bar{q}\right)^{T}, \qquad \theta = atan2\left(v_y,\ v_x\right) \tag{22}$$

The corner of the part then follows from the centroid, the angle and the known part size.

9.2 Touch probe (fine)

The probe starts 3 mm before the edge predicted by the vision estimate and moves up to 6 mm until it touches (repeatability 2 µm). Three touch points q_1, q_2, q_3 on the long edge are fitted with a total-least-squares line: the line through their mean $\bar{q}$ whose normal n minimises the squared perpendicular distances,

$$\min_{n}\sum_{k}\left(n^{T}\left(q_k - \bar{q}\right)\right)^2, \qquad |n| = 1 \tag{23}$$

which is the first principal direction of the points (found with a singular value decomposition). The angle of the line is the part rotation. Two touches on the short edge give a second line perpendicular to the first, and the intersection of the two lines is the corner. The result is judged by the worst error over the four part corners c_k if the part were machined with the estimated pose T_{est} instead of the true pose T_{true}:

$$E = \max_{k}\left|T_{est}\left(c_k\right) - T_{true}\left(c_k\right)\right| \tag{24}$$

10. Simulator and hardware readiness

The 3D simulator (sim/cnc_sim.html, three.js) builds the machine as a chain of transforms that mirrors equation (1): an A group rotating about x contains a C group rotating about z, which holds the table, the dome and the cut path; the spindle group moves by (X, Y, Z). Playing the planned joint values therefore shows the kinematics working: the tool stays on the dome and normal to it. For a real machine, the G-code uses joint coordinates with G93, which a LinuxCNC or grblHAL controller can run. The servo firmware turns an Arduino into a closed-loop drive for one DC-motor axis: STEP/DIR inputs give the target, quadrature interrupts count the encoder, and the same PID + feedforward law runs at 1 kHz with a following-error fault.

11. Workflow

1. Locate the part: camera → coarse pose → guided probing → fine pose (work offset).
2. Generate the toolpath in part coordinates; run inverse kinematics; check collisions and limits.
3. Plan move times under joint limits; write G93 G-code; verify by reading it back.
4. Execute: interpolator → PID + feedforward servos → machine, while the ML monitor adapts speed and feed.
5. Evaluate every stage against a baseline (run_all.py writes all figures and metrics used in this report).

12. Individual contribution

This is an individual project. [STUDENT NAME] ([REG. NO]) carried out all of the work:

Area	Work done
Machine design	Chose the table-tilting layout, axis limits, tool and holder geometry
Kinematics	Derived forward and inverse kinematics, solution selection, C unwrapping and singularity handling; verified by round trip
Motion planning	Dome spiral CAM, holder collision check, velocity- and acceleration-limited feed planner, G93 G-code writer and reader
Control	Axis models with friction, gravity and cutting load; interpolator; P / PID / PID + feedforward; pole-placement tuning; bandwidth study
AI / ML	Signal model, feature design, rules vs decision tree vs random forest, adaptive machining logic
Perception	Otsu and moments vision pipeline, guided touch probing with least-squares fitting, accuracy study
Simulation and hardware	3D three.js simulator, Arduino servo firmware, hardware guide, learning guide, this report

IMPLEMENTATION – CODING AND OUTPUT SCREENSHOT

The project is written in Python 3 (NumPy, SciPy, scikit-learn, Matplotlib), JavaScript with three.js (3D simulator) and Arduino C++ (servo firmware). The listings below are the core functions, copied directly from the source files; plotting code is left out. Running python3 run_all.py regenerates every result, figure, the G-code and the simulator data in about 40 seconds.

A. Coding

Listing 1: Rotations, forward and inverse kinematics (kinematics.py)

```
def rot_x(deg):
    a = np.radians(deg)
    return np.array([[1, 0, 0], [0, np.cos(a), -np.sin(a)], [0, np.sin(a), np.cos(a)]])


def rot_z(deg):
    c = np.radians(deg)
    return np.array([[np.cos(c), -np.sin(c), 0], [np.sin(c), np.cos(c), 0], [0, 0, 1]])


def table_rotation(A, C):
    """Rotation that takes a point fixed on the part to machine coordinates:
    first spin with the C table, then tilt with the A trunnion."""
    return rot_x(A) @ rot_z(C)


def forward(q):
    """Joints (X, Y, Z, A, C) -> tool tip p and tool direction u in PART coordinates."""
    X, Y, Z, A, C = q
    R = table_rotation(A, C)
    p = R.T @ np.array([X, Y, Z])            # undo the table rotation
    u = R.T @ np.array([0.0, 0.0, 1.0])      # the tool points along machine +Z
    return p, u


def inverse(p, u, prev=None):
    """Tool tip p and direction u in PART coordinates -> joints (X, Y, Z, A, C).


    From forward(): u = (sinA sinC, sinA cosC, cosA), so
        A = +-acos(u_z),   C = atan2(u_x, u_y)  (C shifts by 180 deg when A is negative).
    There are two solutions; we keep the one inside the A limits that is closest
    to the previous pose, so the machine never makes a needless half-turn.
    At A = 0 (tool vertical) C can be anything: this is the SINGULARITY, and we
    simply keep the previous C."""
    u = np.asarray(u, float) / np.linalg.norm(u)
    A = np.degrees(np.arccos(np.clip(u[2], -1, 1)))
    if A < 1e-6:                                        # singular: C undefined
        C = prev[4] if prev is not None else 0.0
        candidates = [(0.0, C)]
    else:
        C = np.degrees(np.arctan2(u[0], u[1]))
        candidates = [(A, C), (-A, C + 180.0)]
    best = None
    for a, c in candidates:
        if not LIMITS["A"][0] <= a <= LIMITS["A"][1]:
            continue
        if prev is not None:                            # unwrap C next to the previous value
            c = prev[4] + ((c - prev[4] + 180) % 360 - 180)
        cost = 0 if prev is None else abs(a - prev[3]) + abs(c - prev[4])
        if best is None or cost < best[0]:
            best = (cost, a, c)
    if best is None:
        raise ValueError(f"tool direction {u} needs A = {A:.1f} deg, outside the A limits")
    _, A, C = best
    X, Y, Z = table_rotation(A, C) @ np.asarray(p, float)
    return np.array([X, Y, Z, A, C])


def limit_violations(q):
    """Names of the joints outside their travel limits."""
    return [ax for ax, v in zip(AXES, q) if not LIMITS[ax][0] <= v <= LIMITS[ax][1]]
```

Listing 2: Dome toolpath, collision check, inverse kinematics along the path, G-code writer and reader (toolpath.py)

```
def surface_height(x, y):
    """Height of the part (or table, 0) at a table position (part coordinates)."""
    r2 = (x - OFFSET[0]) ** 2 + (y - OFFSET[1]) ** 2
    return np.where(r2 < BASE_R ** 2, np.sqrt(np.maximum(R_SPHERE ** 2 - r2, 0)) - H_CENTRE, 0.0)


def dome_spiral(phi_max_deg=60.0, turns=12, step=0.5):
    """Spiral over the dome. phi is the angle from the top of the dome, theta the
    angle around it. Returns ball-centre points P (N, 3) and tool axes U (N, 3)."""
    phi_max = np.radians(phi_max_deg)
    pts, phi = [], 0.0
    while phi <= phi_max:
        theta = 2 * np.pi * turns * phi / phi_max
        n = np.array([np.sin(phi) * np.cos(theta), np.sin(phi) * np.sin(theta), np.cos(phi)])
        pts.append(n)
        # advance phi so the step along the surface is about `step` mm
        ds_dphi = R_SPHERE * np.hypot(1, 2 * np.pi * turns / phi_max * np.sin(phi))
        phi += step / ds_dphi
    U = np.array(pts)
    S = U * R_SPHERE + [0, 0, -H_CENTRE] + OFFSET         # contact point on the surface
    P = S + BALL_R * U                                     # ball centre
    return P, U


def holder_clearance(p, u, n=16):
    """Smallest gap (mm) between the tool-holder rim and the part/table below it.
    Negative means the holder would hit something."""
    c = p + STICKOUT * u
    e1 = np.cross(u, [1, 0, 0] if abs(u[0]) < 0.9 else [0, 1, 0]); e1 /= np.linalg.norm(e1)
    e2 = np.cross(u, e1)
    t = np.linspace(0, 2 * np.pi, n, endpoint=False)
    rim = c + HOLDER_R * (np.outer(np.cos(t), e1) + np.outer(np.sin(t), e2))
    return float(np.min(rim[:, 2] - surface_height(rim[:, 0], rim[:, 1])))


def to_joints(P, U):
    """Inverse kinematics for every point, keeping the joints continuous."""
    Q, prev = [], None
    for p, u in zip(P, U):
        prev = kin.inverse(p, u, prev)
        Q.append(prev)
    return np.array(Q)


def write_gcode(Q, seg_times, path):
    """G-code for a real 5-axis controller (e.g. LinuxCNC with trivial kinematics).
    G93 = inverse-time feed: F = 1 / (minutes for this move). The planner already
    knows how long each move must take, and inverse time is the standard way to
    give mixed linear + rotary moves a feed rate."""
    L = ["(5-axis dome finishing, ball-end R3, generated by toolpath.py)",
         "G21 G90 G94 (mm, absolute, feed per minute)",
         "G0 Z%.3f" % SAFE_Z,
         "G0 X%.3f Y%.3f A%.3f C%.3f" % (Q[0][0], Q[0][1], Q[0][3], Q[0][4]),
         "G1 Z%.3f F300" % Q[0][2],
         "G93 (inverse-time feed)"]
    for q, t in zip(Q[1:], seg_times):
        L.append("G1 X%.3f Y%.3f Z%.3f A%.3f C%.3f F%.2f" % (*q, 60.0 / t))
    L += ["G94", "G0 Z%.3f" % SAFE_Z, "M30"]
    open(path, "w").write("\n".join(L) + "\n")
    return len(L)


def read_gcode(path):
    """Read back the G1 moves made in G93 mode -> joints (N, 5) and move times (s)."""
    Q, T, q, inv = [], [], dict(X=0, Y=0, Z=0, A=0, C=0), False
    for line in open(path):
        line = line.split("(")[0].strip()
        if not line:
            continue
        words = {w[0]: float(w[1:]) for w in line.split() if w[0] in "XYZACFGM"}
        inv = True if "G93" in line else False if "G94" in line else inv
        q.update({k: v for k, v in words.items() if k in "XYZAC"})
        if line.startswith("G1") and inv:
            Q.append([q[a] for a in "XYZAC"]); T.append(60.0 / words["F"])
    return np.array(Q), np.array(T)
```

Listing 3: Feed-rate planner (motion_planner.py)

```
def joint_rates(Q, t):
    """Joint velocity of every move (N-1, 5) and acceleration at every junction,
    including starting from and stopping at rest."""
    v = np.diff(Q, axis=0) / t[:, None]
```

```
    vz = np.vstack([np.zeros(5), v, np.zeros(5)])
    tt = np.concatenate([[t[0]], t, [t[-1]]])
    a = np.abs(np.diff(vz, axis=0)) / ((tt[:-1] + tt[1:]) / 2)[:, None]
    return v, a


def plan(Q, P, feed=FEED, use_vel=True, use_acc=True, iters=500):
    """Time (s) for each move between consecutive points."""
    t = np.linalg.norm(np.diff(P, axis=0), axis=1) / feed
    if use_vel:
        t = np.maximum(t, (np.abs(np.diff(Q, axis=0)) / kin.VMAX).max(axis=1))
    if use_acc:
        for _ in range(iters):
            _, a = joint_rates(Q, t)
            r = (a / kin.AMAX).max(axis=1)              # worst axis at each junction
            if r.max() <= 1.001:
                break
            s = np.sqrt(np.maximum(r, 1.0))             # accel ~ 1/t^2, so stretch by sqrt
            t = t * np.maximum(s[:-1], s[1:])           # each move touches two junctions
    return t


def report(name, Q, P, t):
    v, a = joint_rates(Q, t)
    d = np.linalg.norm(np.diff(P, axis=0), axis=1)
    return dict(name=name, cycle_s=float(t.sum()),
                max_speed_ratio=float((np.abs(v) / kin.VMAX).max()),
                max_accel_ratio=float((a / kin.AMAX).max()),
                worst_axis_speed=kin.AXES[int(np.argmax((np.abs(v) / kin.VMAX).max(axis=0)))],
                mean_feed=float(d.sum() / t.sum()))
```

Listing 4: Pole-placement gains, loads, interpolator and servo simulation (axis_control.py)

```
def gains(wn_hz):
    wn = 2 * np.pi * wn_hz
    kp = M * wn ** 2
    kd = np.maximum(2 * ZETA * wn * M - B, 0)
    return kp, kp * wn / 10, kd


def load(q, v, t, rng_phase):
    """Disturbance on each axis: cutting force on X/Y (15 N, varying), gravity on Z
    (4 kg) and the unbalanced trunnion on A (3 kg at 20 mm)."""
    d = np.zeros(5)
    d[0] = 15 * np.sin(2 * np.pi * 0.7 * t + rng_phase)
    d[1] = 15 * np.cos(2 * np.pi * 0.7 * t + rng_phase)
    d[2] = 4.0 * 9.81
    d[3] = 3.0 * 9.81 * 0.02 * np.sin(q[3])
    return d


def interpolate(t_ref, q_ref):
    """The controller's interpolator: straight lines between the planned points,
    sampled every 1 ms, then a 20 ms moving average that rounds the corners so
    the motors never get an instant change of speed. Returns t and the reference
    position, velocity and acceleration (SI units)."""
    t = np.arange(0, t_ref[-1], DT)
    lin = np.column_stack([np.interp(t, t_ref, q_ref[:, i]) for i in range(5)]) * SCALE
    n = int(SMOOTH / DT)
    pad = np.vstack([np.repeat(lin[:1], n, 0), lin, np.repeat(lin[-1:], n, 0)])
    ker = np.ones(n) / n
    r = np.column_stack([np.convolve(pad[:, i], ker, "same") for i in range(5)])[n:-n]
    rv = np.gradient(r, DT, axis=0)
    ra = np.gradient(rv, DT, axis=0)
    return t, r, rv, ra


def simulate(t_ref, q_ref, mode, wn_hz=25.0, seed=0):
    """Run all five axes along the reference. mode: 'P', 'PID' or 'PID+FF'.
    Returns time, actual joints (mm/deg), reference joints (mm/deg)."""
    rng = np.random.default_rng(seed)
    t, r, rv, ra = interpolate(t_ref, q_ref)
    kp, ki, kd = gains(wn_hz)
    if mode == "P":
        ki, kd = 0 * ki, 0 * kd
    x, v = r[0].copy(), np.zeros(5)
    integ, prev_e, dfilt = np.zeros(5), None, np.zeros(5)
    out = np.empty_like(r)
    phase = rng.uniform(0, 2 * np.pi)
    for k in range(len(t)):
        out[k] = x                                          # actual position at time t[k]
        meas = np.round(x / ENC) * ENC                      # encoder
        e = r[k] - meas
        de = 0 if prev_e is None else (e - prev_e) / DT
        dfilt += 0.3 * (de - dfilt)                         # filtered derivative
        prev_e = e
```

```
        u = kp * e + ki * integ + kd * dfilt
        if mode == "PID+FF":
            u += M * ra[k] + B * rv[k] + FC * np.tanh(rv[k] / 1e-4)
            u[2] += 4.0 * 9.81                              # known weight of the Z axis
        u_sat = np.clip(u, -UMAX, UMAX)
        integ += e * DT * (u == u_sat)                      # anti-windup: stop integrating when
saturated
        acc = (u_sat - B * v - FC * np.tanh(v / 1e-4) - load(x, v, t[k], phase)) / M
        v += acc * DT
        x += v * DT
    return t, out / SCALE, r / SCALE


def errors(t, q, r):
    """Following error per axis and contour error of the tool on the part."""
    fe = q - r
    Pa = np.array([kin.forward(x)[0] for x in q[::5]])
    Pr = np.array([kin.forward(x)[0] for x in r])
    dist, _ = cKDTree(Pr).query(Pa)
    return dict(rms_follow=np.sqrt((fe ** 2).mean(0)).tolist(),
max_follow=np.abs(fe).max(0).tolist(),
                rms_contour_um=float(1000 * np.sqrt((dist ** 2).mean())), max_contour_um=float(1000
* dist.max()),
                fe=fe, contour=dist)
```

Listing 5: Tool-monitoring signals, features, baseline rules and adaptive job (tool_monitor.py)

```
def make_window(state, rng, rpm=None, feed=None, wear=None, material=None):
    """Simulate one window of current and vibration for a tool state. material
    scales the cutting force (soft aluminium ~0.7 ... hard alloy ~1.5) and is NOT
    known to the monitor, which is what makes a fixed current threshold unreliable."""
    rpm = rng.uniform(9000, 15000) if rpm is None else rpm
    material = rng.uniform(0.7, 1.5) if material is None else material
    feed = rng.uniform(0.6, 1.4) if feed is None else feed            # x nominal chip load
    if wear is None:
        wear = rng.uniform(WORN, 1.0) if state == 1 else rng.uniform(0, 0.3)
    t = np.arange(int(FS * WIN)) / FS
    f_rev = rpm / 60
    phase = 2 * np.pi * f_rev * t + rng.uniform(0, 2 * np.pi)
    tooth = np.floor(phase / np.pi).astype(int) % 2                   # which of the 2 teeth is
cutting
    pulse = np.maximum(np.sin(2 * phase), 0)                          # one pulse per tooth pass
    force = 100 * material * feed * (1 + 0.8 * wear) * pulse
    if state == 3:                                                    # broken tooth: one tooth cuts
nothing,
        force = np.where(tooth == 0, 0.0, 1.8 * force)                # the other takes almost
double
    fn = rng.uniform(1300, 1700)                                      # structural resonance
    b, a = signal.iirpeak(fn, 8, FS)
    vib = 0.02 * signal.lfilter(b, a, force) + 0.002 * force
    vib += rng.normal(0, 0.05 + 0.25 * wear, len(t))                  # sensor noise + rubbing of
the worn edge
    if state == 2:                                                    # chatter: growing self-
excited vibration
        grow = np.minimum(1, 0.3 + t / WIN)
        vib += rng.uniform(4, 8) * grow * np.sin(2 * np.pi * fn * rng.uniform(0.98, 1.02) * t)
        force = force * (1 + 0.15 * np.sin(2 * np.pi * fn * t))
    smooth = signal.lfilter(*signal.butter(2, 50, fs=FS), force)
    current = IDLE_CURRENT + 0.04 * smooth + rng.normal(0, 0.05, len(t))
    return current, vib, rpm, feed


def features(current, vib, rpm, feed):
    """8 features a person can interpret."""
    f, P = signal.periodogram(vib, FS)
    tot = P.sum() + 1e-12
    band = lambda lo, hi: P[(f >= lo) & (f < hi)].sum()
    f_rev, f_tp = rpm / 60, 2 * rpm / 60
    near = lambda fc: band(fc - 15, fc + 15)
    half = len(current) // 2                                           # skip filter start-up
    return np.array([feed, (current[half:].mean() - IDLE_CURRENT) / feed, current[half:].std(),
np.sqrt(np.mean(vib ** 2)),
                     stats.kurtosis(vib), band(1100, 1900) / tot, near(f_tp) / tot,
                     near(f_rev) / (near(f_tp) + 1e-12)])


class Rules:
    """Hand-written thresholds, the 'no machine learning' baseline."""


    def predict(self, X):
        out = np.zeros(len(X), int)
        out[X[:, 1] > 1.65] = 1               # high cutting current per unit feed -> worn
        out[X[:, 5] > 0.5] = 2                # vibration energy at resonance -> chatter
        out[X[:, 7] > 0.5] = 3                # strong once-per-rev pulse -> broken tooth
```

```
        return out


def job_demo(model):
    """A 120 s job in one material: the tool wears steadily, the start speed is in a chatter
    zone, and a tooth breaks at 100 s. The monitor reacts once per second."""
    rng = np.random.default_rng(5)
    rpm, feed, stopped, log = 12000.0, 1.0, False, []
    chatter_zone = (11000, 13000)
    for s in range(120):
        wear = s / 120 * 0.9
        if stopped:
            log.append((s, rpm, feed, wear, -1, -1, "stopped")); continue
        state = 3 if s >= 100 else 2 if chatter_zone[0] < rpm < chatter_zone[1] and feed > 0.7 else
1 if wear > WORN else 0
        x = features(*make_window(state, rng, rpm, feed, wear, material=1.2))
        pred = int(model.predict(x[None])[0])
        used = (rpm, feed)                                      # settings during this window
        action = "-"
        if pred == 2:
            rpm *= 1.15; action = "chatter: spindle speed +15%"
        elif pred == 1 and feed > 0.8:
            feed *= 0.8; action = "worn: feed -20%, tool change flagged"
        elif pred == 3:
            stopped = True; action = "broken tooth: feed hold, spindle stop"
        log.append((s, *used, wear, state, pred, action))
    return log
```

Listing 6: Vision and touch-probe localisation (probing.py)

```
def otsu(img):
    """Threshold that best separates dark and bright pixels (max between-class variance)."""
    h, edges = np.histogram(img, 256, (0, 1))
    p = h / h.sum(); c = (edges[:-1] + edges[1:]) / 2
    w0 = np.cumsum(p); m = np.cumsum(p * c); mt = m[-1]
    var = (mt * w0 - m) ** 2 / (w0 * (1 - w0) + 1e-12)
    return c[np.argmax(var)]


def vision(img):
    """Coarse pose from one image: threshold -> clean -> biggest blob -> moments."""
    mask = ndimage.binary_opening(img > otsu(img), iterations=3)          # removes the chips
    lab, n = ndimage.label(mask)
    blob = lab == 1 + np.argmax(ndimage.sum(mask, lab, range(1, n + 1)))
    ys, xs = np.nonzero(blob)
    x, y = (xs + .5) / PX - FOV / 2, (ys + .5) / PX - FOV / 2
    c = np.array([x.mean(), y.mean()])
    cov = np.cov(np.vstack([x - c[0], y - c[1]]))
    evals, evecs = np.linalg.eigh(cov)
    d = evecs[:, 1]                                                       # direction of the long
side
    ang = np.degrees(np.arctan2(d[1], d[0]))
    ang = (ang + 90) % 180 - 90                                           # a line has no front/back
    corner = c - rot(ang) @ np.array([L / 2, W / 2])
    return np.array([*corner, ang]), blob


def probe_touch(start, direction, pose, rng):
    """Move from start along direction for up to 2*SEARCH mm; return the touch point or None."""
    s0, d = to_part([start], pose)[0], direction @ rot(pose[2])          # work in part coordinates
    hits = []
    for axis, val in ((1, 0.0), (0, 0.0)):                                # bottom edge y=0, left
edge x=0
        if abs(d[axis]) < 1e-9:
            continue
        s = (val - s0[axis]) / d[axis]
        q = s0 + s * d
        other = q[1 - axis]
        if 0 <= s <= 2 * SEARCH and 0 <= other <= (L if axis == 1 else W):
            hits.append(s)
    if not hits:
        return None
    s = min(hits) + rng.normal(0, PROBE_SIGMA)
    return np.asarray(start) + s * np.asarray(direction)


def probe(prior, pose, rng, n_long=3, n_short=2):
    """Probe n_long points on the long edge and n_short on the short edge, starting
    from where the prior pose says the edges are. Returns the fitted pose or None (missed)."""
    R = rot(prior[2])
    long_pts, short_pts = [], []
    for f in np.linspace(0.15, 0.85, n_long):                              # along the bottom edge,
moving +y'
        exp = to_table([[f * L, 0]], prior)[0]
        d = R @ np.array([0, 1.0])
        p = probe_touch(exp - SEARCH * d, d, pose, rng)
```

```
        if p is None:
            return None, None
        long_pts.append(p)
    for f in np.linspace(0.3, 0.7, n_short):                                  # along the left edge,
moving +x'
        exp = to_table([[0, f * W]], prior)[0]
        d = R @ np.array([1.0, 0])
        p = probe_touch(exp - SEARCH * d, d, pose, rng)
        if p is None:
            return None, None
        short_pts.append(p)
    A, B = np.array(long_pts), np.array(short_pts)
    # least-squares line through the long-edge points (principal direction)
    ca = A.mean(0)
    u = np.linalg.svd(A - ca)[2][0]
    u = u if u[0] > 0 else -u
    ang = np.degrees(np.arctan2(u[1], u[0]))
    # the short edge is perpendicular to it (rectangular block) and passes through the mean of its
points
    cb = B.mean(0)
    nrm = u                                                                   # short edge: points p
with (p - cb).u = 0
    t = (cb - ca) @ nrm                                                       # corner = ca + t u
    corner = ca + t * u
    return np.array([*corner, ang]), (A, B)


def part_error(est, pose):
    """Worst position error (mm) over the part's four corners if we machine with
    the estimated pose instead of the true one."""
    c = np.array([[0, 0], [L, 0], [L, W], [0, W]])
    return float(np.max(np.linalg.norm(to_table(c, est) - to_table(c, pose), axis=1)))
```

Listing 7: 3D simulator: kinematic chain and animation (sim/cnc_sim.html)

```
// A trunnion (tilts about X) -> C table (spins about Z) -> part
const A = new THREE.Group(); M.add(A);
const cradle = box(230, 120, 12, 0x6b7280); cradle.position.z = -40; A.add(cradle);
for (const s of [-1, 1]) { const w = box(12, 120, 40, 0x6b7280); w.position.set(s * 109, 0, -20);
A.add(w); }
const C = new THREE.Group(); A.add(C);
const table = cylZ(80, 14, 0x4b5563); table.position.z = -7; C.add(table);
for (let k = 0; k < 4; k++) { const sl = box(150, 3, 1, 0x374151); sl.rotation.z = k * Math.PI / 4;
sl.position.z = 0.2; C.add(sl); }
const cap = Math.acos(TP.H / TP.R);
const domeGeo = new THREE.SphereGeometry(TP.R, 64, 32, 0, Math.PI * 2, 0, cap);
const dome = new THREE.Mesh(domeGeo, mat(0xd6dde6, { metalness: 0.5, roughness: 0.35 }));
dome.rotation.x = Math.PI / 2; dome.position.set(TP.offset[0], TP.offset[1], -TP.H); C.add(dome);
const trailGeo = new THREE.BufferGeometry();
const trailPos = new Float32Array(TP.p.length * 3);
TP.p.forEach((p, i) => { trailPos.set([p[0], p[1], p[2] - TP.ball + 0.15], 3 * i); });
trailGeo.setAttribute('position', new THREE.BufferAttribute(trailPos, 3));
const trail = new THREE.Line(trailGeo, new THREE.LineBasicMaterial({ color: 0xdc2626 }));
C.add(trail);


function frame(now) {
  const dt = (now - last) / 1000; last = now;
  if (playing) { t += dt * +$('spd').value; if (t > T_END) t = 0; }
  const { q, i, j } = sample(t);
  S.position.set(q[0], q[1], q[2]);
  A.rotation.x = rad(q[3]);
  C.rotation.z = rad(q[4]);
  trailGeo.setDrawRange(0, j + 1);
  const pi = TP.p[i], pj = TP.p[j];
  const feed = Math.hypot(pj[0] - pi[0], pj[1] - pi[1], pj[2] - pi[2]) / ((TP.t[j] - TP.t[i]) || 1);
  ['X', 'Y', 'Z'].forEach((n, k) => $('j' + n).textContent = q[k].toFixed(2) + ' mm');
  $('jA').textContent = q[3].toFixed(2) + ' deg'; $('jC').textContent = q[4].toFixed(1) + ' deg';
  $('feed').textContent = feed.toFixed(1) + ' mm/s';
  $('sing').textContent = Math.abs(q[3]) < 8 ? 'Near the singularity (A < 8 deg): C spins fast, feed
reduced' : '';
  $('tv').textContent = t.toFixed(1) + ' s / ' + T_END.toFixed(0) + ' s';
  $('scrub').value = Math.round(t / T_END * 1000);
  placeCamera();
  renderer.render(scene, camera);
  requestAnimationFrame(frame);
}
```

Listing 8: Servo axis firmware for the real machine (firmware/axis_servo_pid/axis_servo_pid.ino)

```
// One closed-loop servo axis for the 5-axis CNC (build five of these).
// Arduino Uno + DC gear motor with quadrature encoder + H-bridge (BTS7960 or L298N).
//
```

```
// The CNC controller (LinuxCNC or grblHAL) sends STEP/DIR pulses, exactly as it
// would to a stepper driver. This board counts them as the target position,
// reads the motor encoder as the actual position, and runs the same
// PID + feedforward law as axis_control.py at 1 kHz.
//
// Wiring
//   Encoder A -> D2 (INT0), Encoder B -> D3 (INT1), encoder 5V/GND
//   STEP in   -> D8 (pin-change interrupt), DIR in -> D4, controller GND -> GND
//   H-bridge: PWM -> D9, DIR -> D7, ENABLE -> D6
//   Fault LED -> D13 (on if the following error grows too large)


const byte ENC_A = 2, ENC_B = 3, STEP_IN = 8, DIR_IN = 4;
const byte PWM_OUT = 9, DIR_OUT = 7, EN_OUT = 6, FAULT_LED = 13;


// Gains in PWM per encoder count. Start values for a small 12 V gear motor;
// tune with the 's' step test on the Serial Monitor (see docs/HARDWARE.md).
const float KP = 4.0;            // proportional
const float KI = 40.0;           // integral (per second)
const float KD = 0.05;           // derivative (seconds)
const float KV = 0.0;            // velocity feedforward: PWM per (count/s), set after a speed test
const float D_ALPHA = 0.3;       // low-pass on the derivative
const float I_LIMIT = 200;       // anti-windup clamp on KI * integral (PWM)
const long  MAX_FOLLOW = 2000;   // following-error limit (counts) -> fault, motor off
const unsigned long PERIOD_US = 1000;   // 1 kHz servo loop


volatile long encCount = 0;      // actual position (counts)
volatile long target = 0;        // commanded position (step pulses)
float integ = 0, dFilt = 0;
long prevErr = 0, prevTarget = 0;
bool fault = false;
unsigned long nextTick;


// Quadrature decoding: on every edge of A or B, compare the two channels
void encA() { encCount += (digitalRead(ENC_A) == digitalRead(ENC_B)) ? 1 : -1; }
void encB() { encCount += (digitalRead(ENC_A) != digitalRead(ENC_B)) ? 1 : -1; }


// STEP input on D8 = PCINT0: count one step on each rising edge, direction from DIR
ISR(PCINT0_vect) {
  if (PINB & _BV(PB0)) target += (PIND & _BV(PD4)) ? 1 : -1;
}


void motor(float u) {
  int pwm = constrain((int)fabs(u), 0, 255);
  digitalWrite(DIR_OUT, u >= 0);
  analogWrite(PWM_OUT, pwm);
}


void setup() {
  pinMode(ENC_A, INPUT_PULLUP); pinMode(ENC_B, INPUT_PULLUP);
  pinMode(STEP_IN, INPUT); pinMode(DIR_IN, INPUT);
  pinMode(PWM_OUT, OUTPUT); pinMode(DIR_OUT, OUTPUT); pinMode(EN_OUT, OUTPUT); pinMode(FAULT_LED,
OUTPUT);
  attachInterrupt(digitalPinToInterrupt(ENC_A), encA, CHANGE);
  attachInterrupt(digitalPinToInterrupt(ENC_B), encB, CHANGE);
  PCICR |= _BV(PCIE0); PCMSK0 |= _BV(PCINT0);      // enable pin-change interrupt on D8
  digitalWrite(EN_OUT, HIGH);
  Serial.begin(115200);
  nextTick = micros();
}


void loop() {
  if ((long)(micros() - nextTick) < 0) return;      // wait for the next 1 ms tick
  nextTick += PERIOD_US;
  const float dt = PERIOD_US * 1e-6;


  noInterrupts(); long pos = encCount, tgt = target; interrupts();
  long err = tgt - pos;


  if (labs(err) > MAX_FOLLOW) fault = true;         // lost position: stop for safety
  if (fault) { motor(0); digitalWrite(EN_OUT, LOW); digitalWrite(FAULT_LED, HIGH); return; }


  dFilt += D_ALPHA * ((err - prevErr) / dt - dFilt);  // filtered derivative of the error
  prevErr = err;
  float vRef = (tgt - prevTarget) / dt;               // commanded speed (counts/s)
  prevTarget = tgt;


  float u = KP * err + KI * integ + KD * dFilt + KV * vRef;
  if (fabs(u) < 255) {                               // anti-windup: integrate only when not
saturated
```

```
    integ += err * dt;
    integ = constrain(integ, -I_LIMIT / KI, I_LIMIT / KI);
  }
  motor(u);


  // Serial: 's' = 1000-count step test, 'z' = zero position, otherwise print every 50 ms
  if (Serial.available()) {
    char c = Serial.read();
    if (c == 's') { noInterrupts(); target += 1000; interrupts(); }
    if (c == 'z') { noInterrupts(); target = encCount = 0; interrupts(); integ = 0; }
  }
  static byte n = 0;
  if (++n >= 50) { n = 0; Serial.print(tgt); Serial.print(','); Serial.print(pos);
Serial.print(','); Serial.println(u, 1); }
}
```

B. Output screenshots

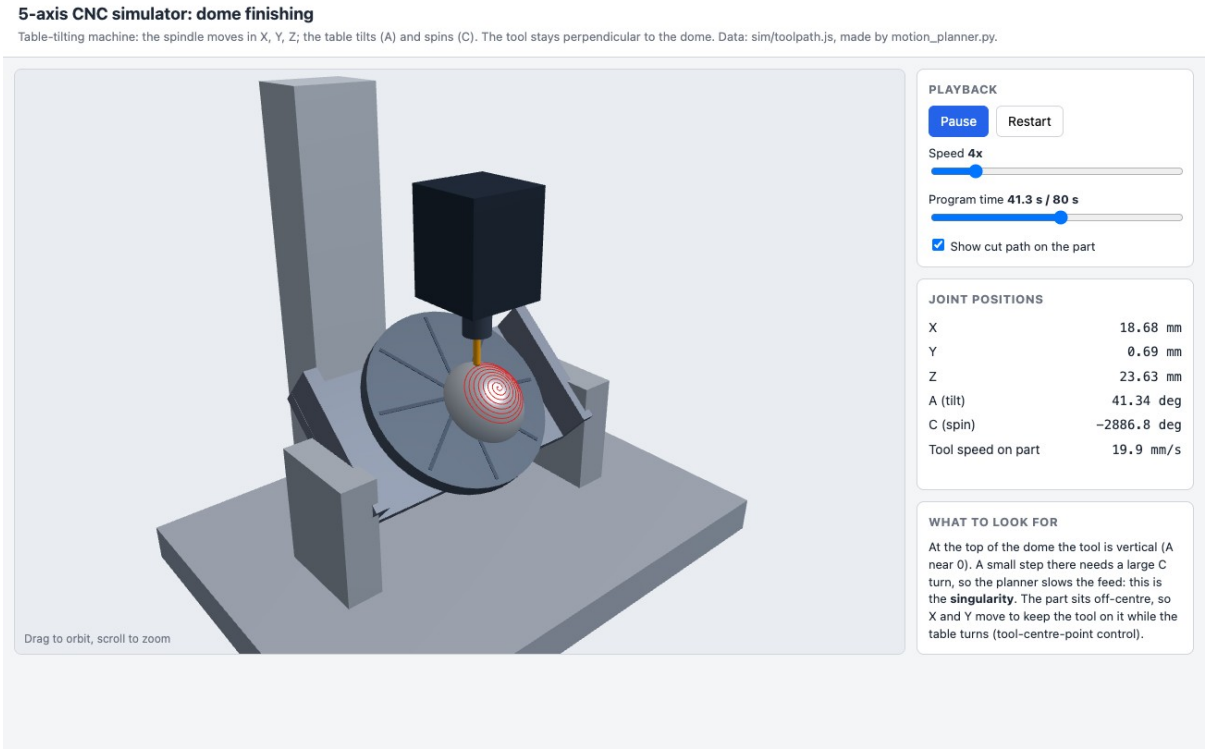


Figure 5: 3D simulator at 43 s: the table has tilted (A = 42°) and turned (C) while the tool stays normal to the dome; the red line is the path cut so far

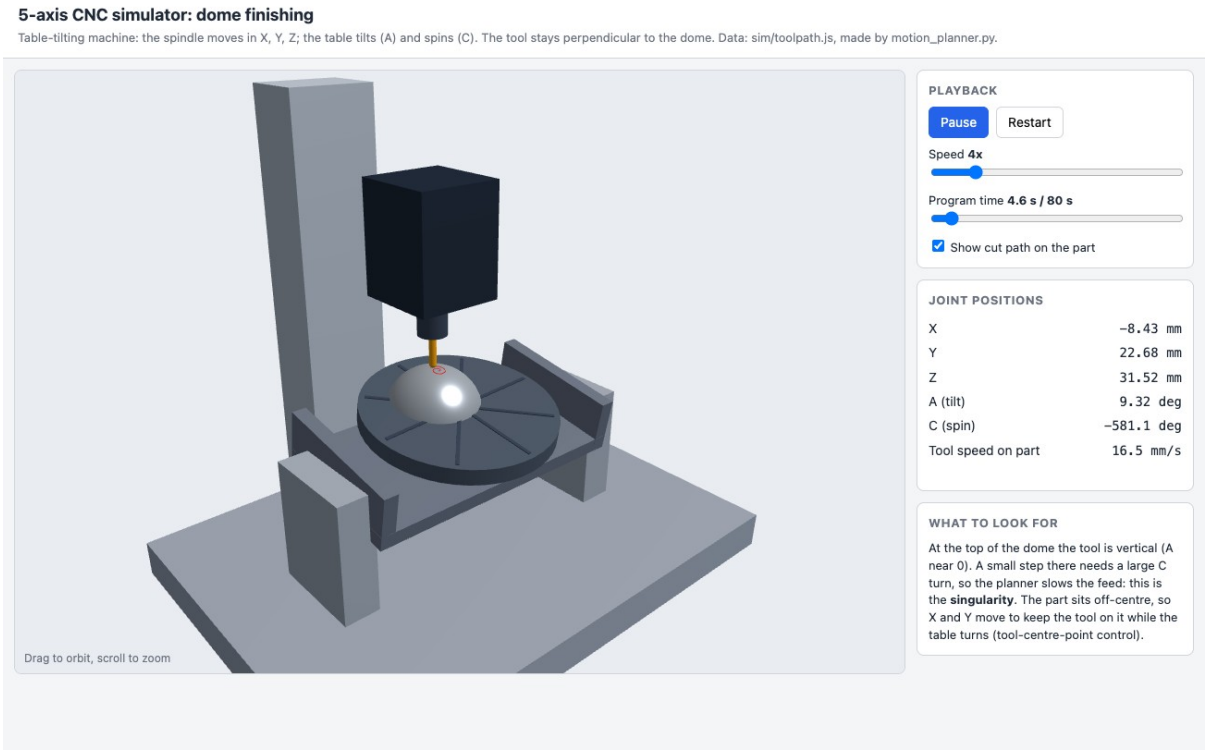


Figure 6: 3D simulator near the start: the tool is almost vertical (A = 9°), close to the singularity, so the planner has reduced the feed

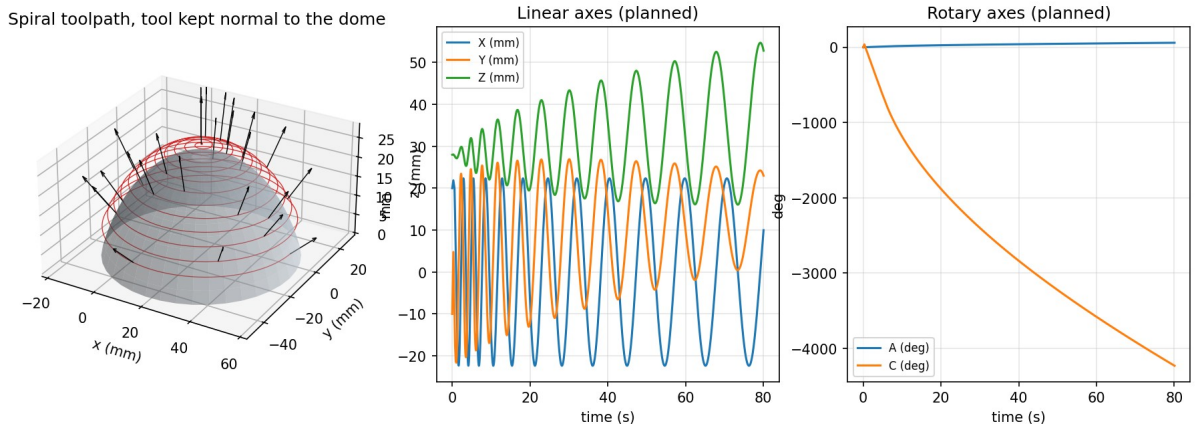


Figure 7: Spiral toolpath with tool axes (left) and the planned linear and rotary axis motion (centre, right)

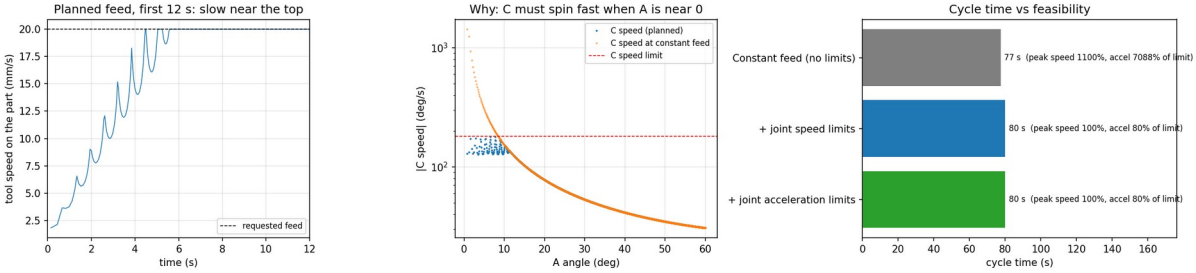


Figure 8: Feed planning: the feed is reduced only near the top of the dome, where C would otherwise spin faster than its limit

Listing 9: First lines of the generated G-code (results/dome_finish.ngc)

```
(5-axis dome finishing, ball-end R3, generated by toolpath.py)
G21 G90 G94 (mm, absolute, feed per minute)
G0 Z45.000
G0 X20.000 Y-10.000 A0.000 C0.000
G1 Z28.000 F300
G93 (inverse-time feed)
G1 X21.883 Y4.786 Z28.059 A0.716 C38.434 F202.89
G1 X20.018 Y-9.634 Z27.786 A1.249 C0.104 F208.04
G1 X13.087 Y-17.696 Z27.489 A1.634 C-27.613 F372.14
G1 X5.120 Y-21.244 Z27.269 A1.947 C-50.199 F376.54
G1 X-2.444 Y-21.630 Z27.151 A2.218 C-69.709 F396.63
G1 X-8.985 Y-19.814 Z27.135 A2.460 C-87.126 F458.66
G1 X-14.243 Y-16.517 Z27.210 A2.681 C-103.001 F570.51
G1 X-18.147 Y-12.294 Z27.362 A2.884 C-117.681 F710.35
```

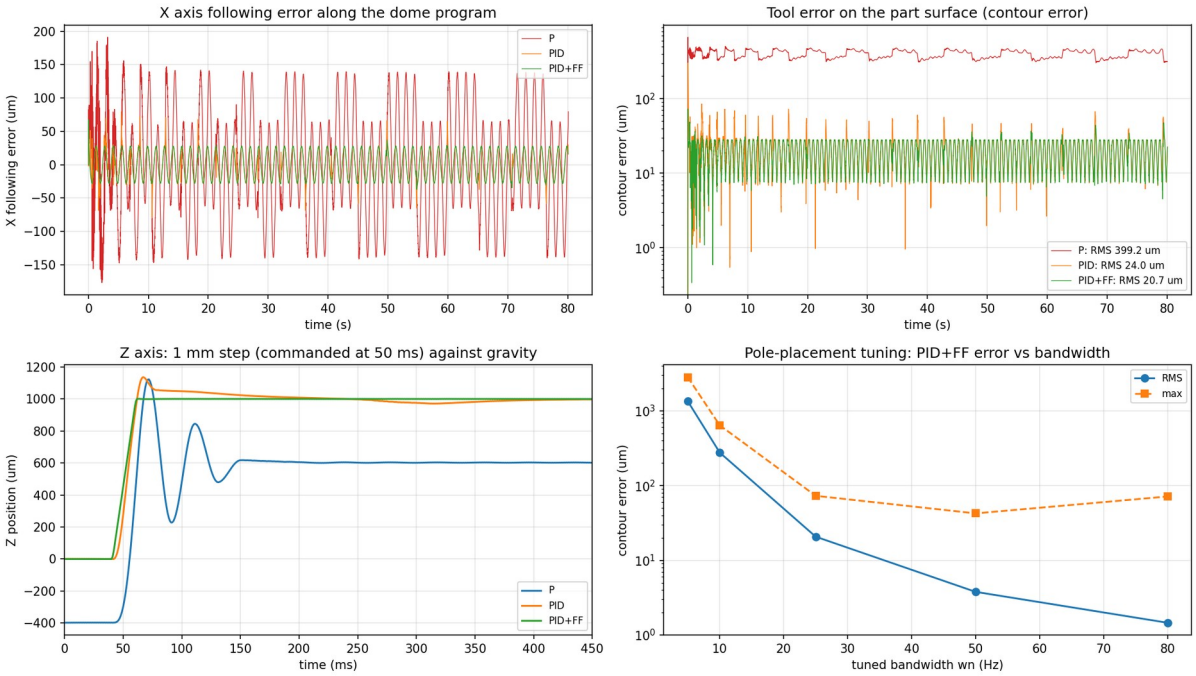


Figure 9: Servo control: X following error, contour error, Z step against gravity and the bandwidth sweep

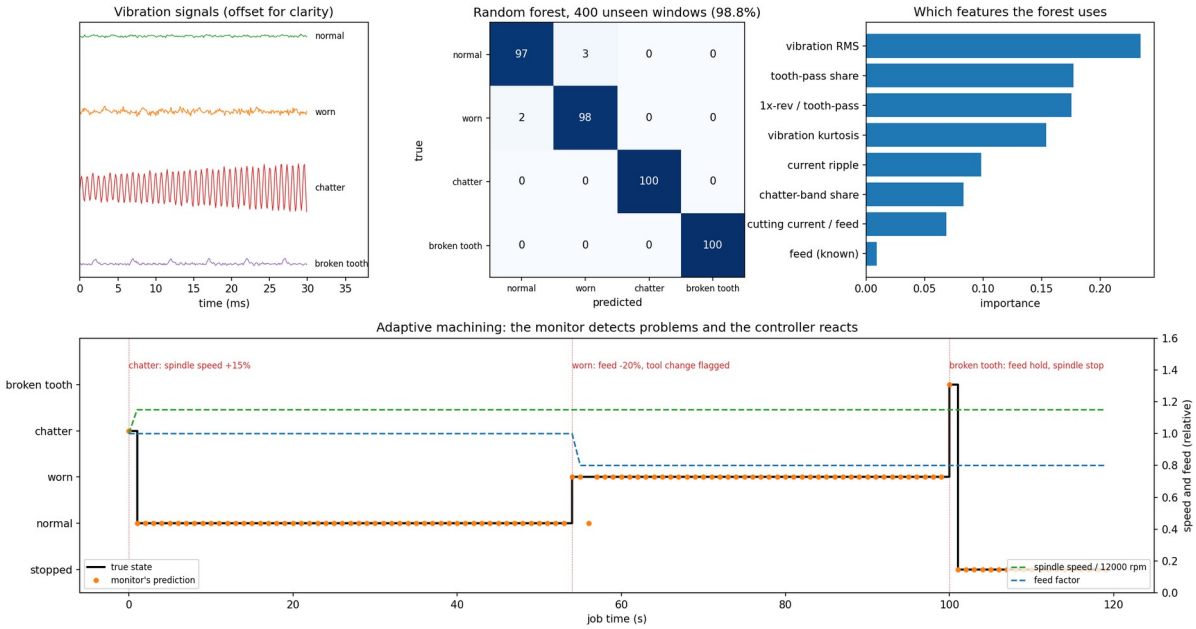


Figure 10: Tool monitoring: vibration signatures, confusion matrix, feature importance and the adaptive job

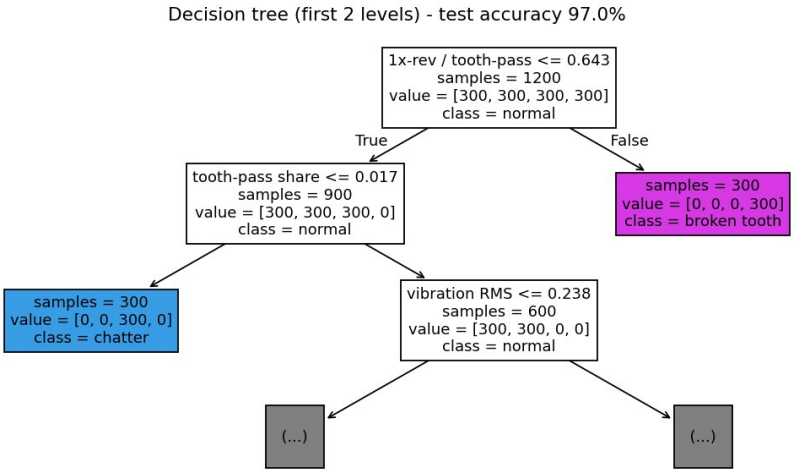


Figure 11: Decision tree learned for tool monitoring (first two levels)

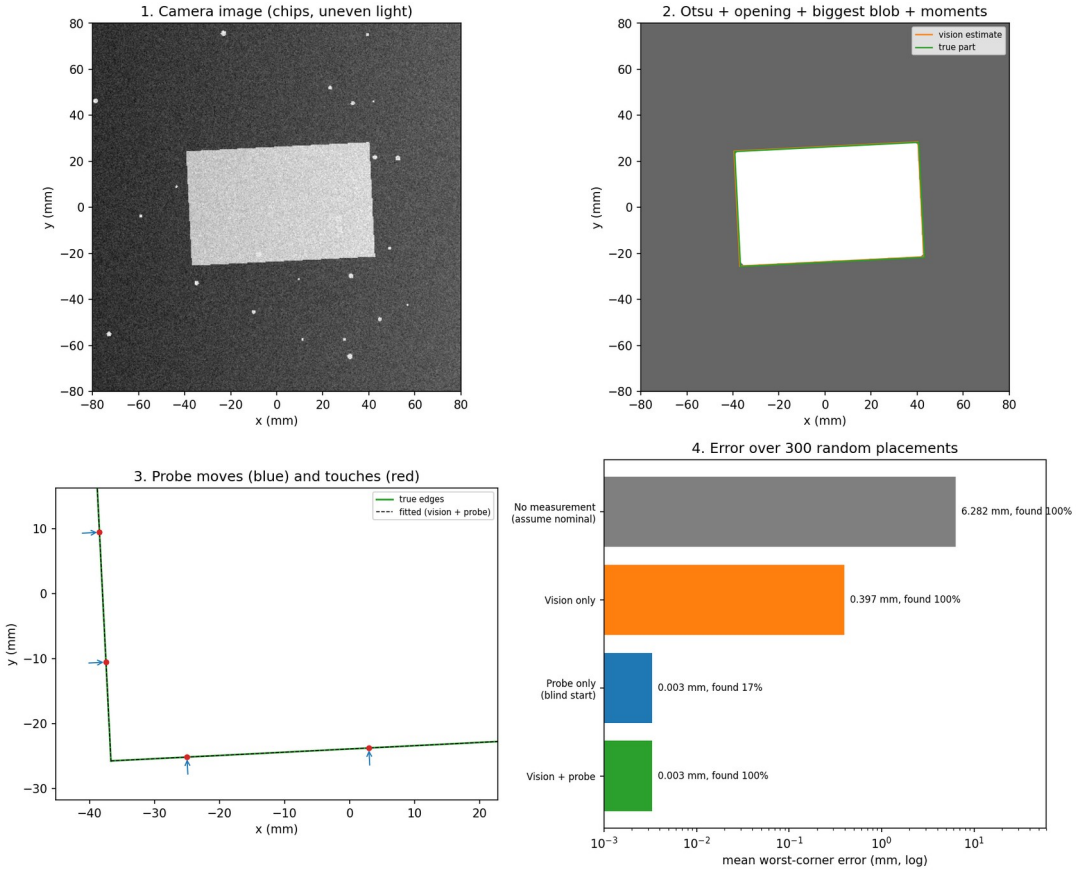


Figure 12: Part localisation: camera image, vision estimate, probe touches and error comparison

C. Results and performance analysis

C.1 Kinematics, planning and G-code

Table 5: Feed planning for the dome program (2882 points)

Plan	Cycle (s)	Mean feed (mm/s)	Peak joint speed / limit	Peak joint accel / limit
Constant feed (no limits)	77.5	20.0	1100% (Y)	7088%
+ joint speed limits	80.1	19.3	100% (X)	80%
+ joint acceleration limits	80.1	19.3	100% (X)	80%

At a constant feed the program would be impossible to run: near the singularity one axis would need 1100% of its top speed and 7088% of its acceleration. Enforcing the joint speed limits fixes both; after it, accelerations peak at 80% of the limit, so the acceleration pass is a safety net that changes nothing for this path. The cost is only 2.6 s (3.4%), because the slow-down is confined to the first few seconds near the top of the dome (Figure 8). The path uses A from 0.0° to 60.0°, the C table turns 4230° in total, there are 0 joint-limit violations, and the holder never comes closer than 8.6 mm to the part. As a check that the collision test works, an 80° tilt just outside the foot of the dome gives -4.5 mm, which is correctly reported as a collision. The 2890-line G-code, read back and passed through forward kinematics, reproduces the tool path to 0.001 mm, the rounding of three decimals in the file.

C.2 Servo control

Table 6: Servo control along the dome program (bandwidth 25 Hz)

Controller	RMS X error (µm)	RMS Z error (µm)	RMS C error (mdeg)	RMS contour (µm)	Max contour (µm)
P	81.4	402.7	85.9	399.2	668.8
PID	20.7	11.9	8.5	24.0	301.9
PID + feedforward	19.7	3.1	2.2	20.7	72.8

Table 7: Z axis: 1 mm step against gravity

Controller	Final error (µm)	Overshoot (µm)
P	-398.4	122.3
PID	-0.7	136.1
PID + feedforward	-0.3	1.3

The P controller lets Z sag 398.4 µm under gravity: it only produces force when there is an error, so it needs an error of mg / K_p to hold the axis up. The integral term builds that force up and removes the sag (0.7 µm), but it overshoots 136.1 µm on a step because the integrator has to unwind. Feedforward supplies the expected force in advance, so the overshoot falls to 1.3 µm and the worst contour error along the program falls from 301.9 to 72.8 µm. The remaining

X and Y error comes from the cutting force, which no model of the axis can predict; feeding back the force estimated by the tool monitor is a natural next step.

Table 8: Effect of the tuned bandwidth (PID + feedforward)

ω_n (Hz)	RMS contour error (µm)	Max contour error (µm)	Comment
5	1343.7	2819.2	too soft
10	276.0	639.7	too soft
25	20.7	72.8	baseline
50	3.8	42.4	best overall
80	1.5	71.3	saturation, noise

Raising the bandwidth makes the axes stiffer against the cutting force: the RMS contour error falls by a factor of about 356 from 5 to 50 Hz. At 80 Hz the RMS falls further but the worst error rises again, because the motors saturate in fast moves and the larger derivative gain amplifies encoder quantisation. About 50 Hz is therefore the best setting for this machine model; it is the optimisation result carried forward to the hardware tuning procedure.

C.3 Machine-learning tool monitoring

Table 9: Tool-state classification on 400 unseen windows with unknown work material

Method	Accuracy	Notes
Hand-written threshold rules	84%	Confuses normal and worn: harder material raises the current just as wear does
Decision tree (depth 4)	97%	Readable if-then rules (Figure 11)
Random forest (100 trees)	98.8%	Top features: vibration RMS, tooth-pass share, 1x-rev / tooth-pass

Table 10: Per-class precision (P), recall (R) and F1: rules against random forest

Class	Rules P	Rules R	Rules F1	Forest P	Forest R	Forest F1
Normal	0.75	0.54	0.63	0.98	0.97	0.97
Worn	0.64	0.82	0.72	0.97	0.98	0.98
Chatter	1.00	1.00	1.00	1.00	1.00	1.00
Broken tooth	1.00	1.00	1.00	1.00	1.00	1.00

Chatter and broken teeth have unmistakable signatures (a large resonance peak; a strong once-per-revolution pulse), so even the rules detect them perfectly. The hard case is wear: a worn tool raises the cutting current, but so does a harder material, which the monitor does not know. The fixed current threshold therefore labels 46 of 100 normal windows as worn. The learned models combine the current with vibration RMS (rubbing of a blunt edge), which responds to wear but not to material, and reach 97% (tree) and 98.8% (forest). The decision tree is almost as accurate and can be read line by line, which matters when an operator has to trust an automatic stop. In the 120 s adaptive job the monitor agreed with the true state in 99% of

seconds: it raised the spindle speed at 0 s to leave the chatter zone, reduced the feed and flagged a tool change at 54 s as the wear passed the limit, and stopped the machine at 100 s when a tooth broke.

C.4 Part localisation

Table 11: Part localisation over 300 random placements (up to 6 mm and 4° off)

Method	Part found	Mean error (mm)	Worst error (mm)
No measurement (assume nominal)	100%	6.282	12.846
Vision only	100%	0.397	0.775
Probe only (blind start)	17%	0.003	0.008
Vision + probe	100%	0.003	0.009

Table 12: Angle error against the number of probe points on the long edge

Probe points on the long edge	Mean angle error (arc-seconds)
2	9.6
3	8.1
5	7.4
8	6.1
12	5.5

Machining at the nominal position would put features 6.3 mm off on average. Vision alone reaches 0.397 mm, limited by the pixel size, the uneven light and the camera calibration error. The probe is more than 100 times more accurate, but starting blind it missed the part in 83% of placements, because the edge was outside its 6 mm search range. Vision-guided probing combines the range of the camera with the accuracy of the probe: 100% success with 0.003 mm error. More probe points average out the 2 µm probe noise, so the angle error falls from 9.6 to 5.5 arc-seconds from 2 to 12 points; three points were chosen as a good balance between accuracy and probing time.

CONCLUSION

An intelligent table-tilting 5-axis CNC machine was designed, simulated and evaluated. Each subsystem uses a method that can be explained step by step, and each was measured against a simpler baseline on test cases that were not used to design it.

- **Kinematics and planning:** closed-form inverse kinematics with solution selection and tool-centre-point compensation is exact to machine precision. The feed planner turned a program that needed 11.0× an axis's top speed into a feasible one for only 3.4% more time. The collision check and the G-code round trip verify the output.
- **Control:** pole-placement PID with feedforward reduced the RMS contour error from 399.2 µm to 20.7 µm and removed the gravity sag. The bandwidth study identified about 50 Hz (3.8 µm) as the best setting.
- **AI / ML:** a random forest on eight physical features recognised tool states with 98.8% accuracy where fixed rules reached 84.0%, and closed the loop by changing speed, feed or stopping the machine.
- **Perception:** vision-guided probing located the part to 0.003 mm in every trial, combining the camera's range with the probe's accuracy.

Mapping to the evaluation rubrics

Rubric	Where it is addressed
Robot design and architecture	Machine layout and specification, system architecture (Methodology 1–3)
Kinematics and motion planning	Forward/inverse kinematics, singularity, toolpath, collision check, feed planning, G-code (Methodology 4–6; Results C.1)
AI / ML algorithm implementation	Decision tree and random forest tool monitoring with feature design (Methodology 8; Results C.3)
Sensor integration and perception	Encoders, current and vibration sensing, camera vision, touch probe (Methodology 7–9; Results C.4)
Robot control and navigation	PID + feedforward servo control, pole placement, adaptive feed and speed (Methodology 7; Results C.2)
Innovation and critical thinking	Coarse-to-fine localisation; ML-driven adaptive machining; baselines and limitations stated
Performance analysis and optimisation	Ablations, bandwidth sweep, probe-point study, per-class metrics (Results C.1–C.4)
Simulation activities	3D simulator, Python simulations of every subsystem, run_all.py

Limitations

- Everything is simulated. Machine dynamics, sensor signals and images come from simple physical models; real gains, thresholds and classifiers must be re-tuned and retrained on measured data.

- The A and C axes are assumed to intersect, and collisions are checked only for the tool holder against the part and the table, not for the whole machine.
- The ball-end tool is kept exactly normal to the surface; real finishing adds a small lead angle so that the tool does not cut with the centre of the ball, where the cutting speed is zero.
- The feed planner limits velocity and acceleration but not jerk; the servo firmware has been syntax-checked but not run on hardware.

Future work

- Build the machine with LinuxCNC (xyzac-trt-kins kinematics with real pivot offsets) and DC servo axes using the provided firmware; identify the real axis masses and friction and re-tune.
- Record real current and vibration data and retrain the monitor; add tool-life prediction (regression on wear) and use the estimated cutting force as feedforward.
- Add lead and tilt angles, full 3D collision checking with machine models, and jerk-limited (S-curve) feed profiles.
- Create a ROS 2 / Gazebo digital twin connected to the real controller.

References

[1] Y. Altintas, Manufacturing Automation: Metal Cutting Mechanics, Machine Tool Vibrations, and CNC Design, 2nd ed., Cambridge University Press, 2012.

[2] B. Siciliano, L. Sciavicco, L. Villani, G. Oriolo, Robotics: Modelling, Planning and Control, Springer, 2009.

[3] K. J. Åström, R. M. Murray, Feedback Systems: An Introduction for Scientists and Engineers, Princeton University Press, 2008.

[4] M. Tomizuka, "Zero phase error tracking algorithm for digital control," ASME J. Dynamic Systems, Measurement, and Control, 109(1), 1987.

[5] R. Teti, K. Jemielniak, G. O'Donnell, D. Dornfeld, "Advanced monitoring of machining operations," CIRP Annals, 59(2), 2010.

[6] L. Breiman, J. Friedman, R. Olshen, C. Stone, Classification and Regression Trees, Wadsworth, 1984.

[7] L. Breiman, "Random forests," Machine Learning, 45(1), 2001.

[8] N. Otsu, "A threshold selection method from gray-level histograms," IEEE Trans. Systems, Man, and Cybernetics, 9(1), 1979.

[9] R. C. Gonzalez, R. E. Woods, Digital Image Processing, 4th ed., Pearson, 2018.

[10] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," JMLR, 12, 2011.

[11] LinuxCNC documentation, "Kinematics" (xyzac-trt-kins) and "G-code: G93 inverse time feed", linuxcnc.org.

EVALUATION SHEET

Reg.No :

Name:

Course code: 24AI2003

Course Name: INTELLIGENT ROBOTICS

S.No	Rubrics	Maximum Marks	Marks Obtained
1	Simulation Activities	10	
2	Intelligent Robotics Project	30	
	Total	40	

S. No.	Rubrics	Maximum Marks	Marks Obtained
1	Robot Design and Architecture	5	
2	Kinematics and Motion Planning	5	
3	AI / ML Algorithm Implementation	5	
4	Sensor Integration and Perception	5	
5	Robot Control and Navigation	5	
6	Innovation and Critical Thinking	4	
7	Performance Analysis and Optimization	4	
8	Presentation and Viva Voce	4	
9	Completeness of the Report	3	
	Total	40	

Performance-Level Assessment
Use the following performance levels to evaluate each rubric.

Rubrics	Excellent	Good	Average	Below Average
Robot Design and Architecture (5)	5	4	3	0–2
Kinematics and Motion Planning (5)	5	4	3	0–2
AI / ML Algorithm Implementation (5)	5	4	3	0–2
Sensor Integration and Perception (5)	5	4	3	0–2
Robot Control and Navigation (5)	5	4	3	0–2
Innovation and Critical Thinking (4)	4	3	2	0–1
Performance Analysis and Optimization (4)	4	3	2	0–1
Presentation and Viva Voce (4)	4	3	2	0–1
Completeness of the Report (3)	3	2	1	0

Level	Description
Excellent	Demonstrates outstanding technical knowledge, accurate implementation, effective integration, and innovative problem-solving.
Good	Demonstrates good understanding and implementation with minor errors or improvements needed.
Average	Demonstrates basic understanding and partial implementation with noticeable limitations.
Below Average	Demonstrates limited understanding, incomplete implementation, and significant technical issues.

Suggested assessment areas: ROS/ROS2, Gazebo or other robotics simulators, sensor data processing, path planning, PID control, computer vision, and robot performance evaluation, depending on the project.

Signature of the Faculty-in-charge